CELESTIAL MECHANICS × AI · 2026

From the next token to the next theorem

How a model works — and how we should use it

Bowen Liu Draft v0.1 45 min

What the model sees: ten integers

EN · ONE SENTENCE

Celestial mechanics is the poetry of orbits.

Cel	estial	_mechanics	_is	_the	_poetry	_of	_or	bits	.
69126	90010	52347	382	290	38153	328	503	24043	13

46 characters → **10** tokens → 10 integers · _ marks a leading space

CHINESE · FOUNDATIONS OF CELESTIAL MECHANICS

本书系统地介绍了天体力学

本	书	系统	地	介绍	了	天	体	力	学
4087	20121	31936	5557	37887	4531	867	7074	11343	8590

12 characters → **10** tokens → 10 integers · two-character words often map to one token

“This book systematically introduces celestial mechanics.”

01 Where do these integers come from?

02 How does it predict the next one?

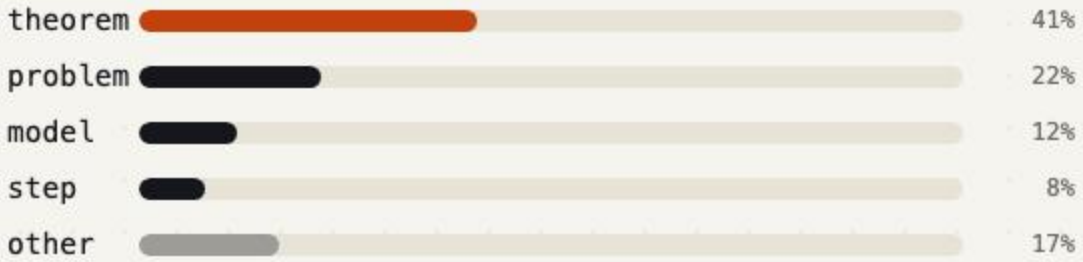
03 How do we put it to work?

Sampling: draw one token from the distribution

Probability distribution · prefix

"One important …"

$$y_{t+1} \sim p_\theta(\cdot \mid x_{1:t})$$



GREEDY

Take the max → **theorem**

SAMPLING

Draw by probability → this time maybe **problem**

KNOBS

temperature · **top-p** control randomness

Input → Transformer → Probability distribution → Sampling → Append to input ↻

> The same prompt can give two different results — that is **sampling**, a **dice roll** by design.

CoT: write the thinking down

Scope: verbal and logical reasoning; arithmetic is a separate step.

Question

A period of 2 years; the semi-major axis grows 4×. What is the new period?

DIRECT ANSWER

8 years ✗ — a number without a plan

COT · PLAN

$T \propto a^{3/2}$; $T = 2 \times 4^{3/2}$

plan ✓ · arithmetic next

TOOL · COMPUTE

```
run_python("2 * 4**1.5") → 16.0 ✓
```

> CoT writes the plan; the tool computes the number.

WHAT COT DOES

Turns words into a plan: parse, recall, set up the relation

WHAT REMAINS

Arithmetic: a separate, tool-friendly step

ESSENCE

Explicit steps; exact numbers are a separate skill

Put the model back in the system



Inner ring: one call · Middle ring: across calls · Outer ring: engineering boundary

01	**LLM**	A probability generator: tokens in, a distribution over the next token out
02	**Prompt**	This call's conditions: system / user / few-shot
03	**Context**	Every token visible to this call; the window is finite
04	**Memory**	State beyond the window: write / retrieve / compress / forget
05	**Agent loop**	Observe → act → observe; state lives outside
06	**Harness**	Tools, sandbox, permissions, verification, logs

> The model is the CPU; prompt, context, memory, loop and harness are the runtime.

Acceleration: from basic use to gold medals

2023 · BEFORE

Basic use

- Explain concepts, search the literature, write code
- Do simple calculations, often wrong
- Complex proofs still need people
- Role: assistant

→ 19 months

2025 · AFTER

Research level

- IMO 2025 gold level 35/42
- Lean proofs, machine-checkable
- OpenAI 2026: 372 problems, ≈3 h each (report)
- Role: candidate generator

2024.07 **28/42** silver → 2025.07 **35/42** gold level — 12 months, +7 points

> This is a staircase with **three steps**.

Timeline: three steps

2023–24

Tools: search + verification

2023.12
FunSearch
LLM + evaluator + evolutionary search; a new cap-set construction

2024.01
AlphaGeometry
Neuro-symbolic; IMO geometry 25/30

2024.07
AlphaProof
Lean + RL; IMO 2024 silver 28/42 (+AG2)

2024–25

Reasoning: RL + test-time compute

2024.09
OpenAI o1
RL teaches thinking; test-time compute

2025.01
DeepSeek-R1
RLVR; reasoning behavior emerges; open weights

2025.05
AlphaEvolve
Evolutionary coding agent; 4×4 complex matrix product in 48 multiplications

2025–26

Scale: from benchmarks to open problems

2025.07
IMO 2025
Gold level 35/42; natural language, no tools

2025
Formal gold (claimed)
Aristotle et al.; Lean-checkable

2026.05–10
OpenAI
Unit-distance counterexample; NS / Riemann-related (claimed); 372 problems (report)

> Every leap combines a **generator, a verifier and search**.

Some guesses: how the big results may have happened

> The following mechanism hypotheses come from public papers, blogs, and benchmarks, and are labeled as **guesses**.

01 DATA

Autoformalization + synthesis: informal problems → Lean; rejection sampling keeps the verifiable ones.

02 TRAINING

RLVR + long CoT: verifiable answers let rewards scale.

03 INFERENCE

Test-time search: best-of-N, tree search, evolution.

04 DISTILLATION

Write search back into the model: less search next time.

05 ENGINEERING

Harness: parallelism, caching, evals, replay.

06 THE BET

Verification: the harder your verifier, the faster your flywheel.

OpenAI: 372 problems, ≈3 hours each (report)

> Three hours per problem looks more like **parallel sampling + verification + search**.

The algorithm engineer's dilemma

It arrived about a year ago. Mathematics is next.

"If the model can write the code, what is my job?"

BEFORE AGENTS

Write the algorithm

- Implement, debug, optimize
- Own the code
- Value = code shipped

→ ~1 year

WITH AGENTS

Direct the search

- Frame the problem
- Design the harness
- Verify and own the system

> The job moves from producing answers to **directing the search**.

From the next token to the next theorem

`token` → `probability` → `candidate` → `verification` → **`theorem`**

Probability gives breadth

Verification gives correctness

People set the direction

> The next theorem still begins with the **next token**.